

# Technical Requirements Document

A practical system design for a secure, observable, asynchronous AI product that keeps business data organised and user-approved.

Version 1.0 | Made by DEX

Prepared from the Project VISTA source brief

## TRD

A coherent operating blueprint for an intelligent startup workspace.

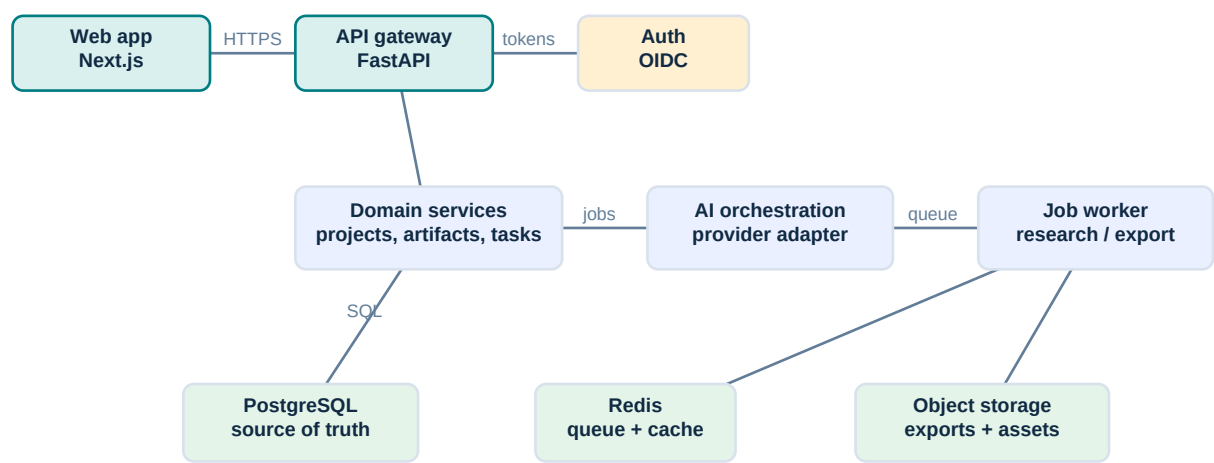
## 1. Technical Direction

The recommended MVP uses a TypeScript Next.js web application, a Python FastAPI service for domain APIs and AI orchestration, PostgreSQL for transactional data, Redis-backed asynchronous work, and S3-compatible object storage. This split keeps the experience fast while making long-running research and generation jobs reliable.

## 2. System Principles

- Organisation-scoped data access by default; project membership further limits access.
- AI is a provider-adapter capability, not embedded product logic; prompts and outputs are versioned.
- Long-running work runs asynchronously and reports progress; requests must remain responsive.
- Every externally sourced claim, artifact revision, and sensitive state change is traceable.
- Interfaces are API-first so a mobile client or integrations can be added later.

## 3. Logical Architecture



Container-level view. Service boundaries can begin as modules in one API deployment and split only when operating load demands it.

## 4. Core Services and Responsibilities

| Component       | Responsibility                                                                  | Key constraint                                               |
|-----------------|---------------------------------------------------------------------------------|--------------------------------------------------------------|
| Web application | Authenticated workspace, editors, review views, exports, notifications.         | No direct database access.                                   |
| API service     | Authorisation, domain rules, CRUD APIs, signed upload URLs, job dispatch.       | Validates organisation and project membership every request. |
| AI orchestrator | Prompt assembly, provider routing, output validation, cost and retry policy.    | Never silently overwrites a user-approved artifact.          |
| Worker service  | Research collection, document rendering, long generations, scheduled reminders. | Idempotent jobs with retry and dead-letter handling.         |
| Data services   | PostgreSQL, Redis, object storage, analytics pipeline.                          | Backups, encryption, retention, access logging.              |

## 5. API and Integration Contract

| Area      | Representative endpoints                                                                   | Contract rules                                                    |
|-----------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Projects  | POST /projects, GET /projects/{id}, PATCH /projects/{id}                                   | JSON; UUID identifiers; cursor pagination.                        |
| Artifacts | POST /projects/{id}/artifacts, POST /artifacts/{id}/generate, POST /artifacts/{id}/approve | Generation returns a job id; approval records actor and revision. |
| Finance   | GET /financial-models/{id}, PATCH /financial-assumptions/{id}                              | Scenario calculations are deterministic and show assumptions.     |
| Tasks     | POST /tasks, PATCH /tasks/{id}, GET /tasks                                                 | Status changes emit an audit event.                               |
| Webhooks  | POST /webhooks/events                                                                      | Signed payloads, retries, idempotency key.                        |

## 6. Non-functional Requirements

| Category      | Requirement                                                                                                                          |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------|
| Security      | TLS in transit; encryption at rest; OIDC; least-privilege service identities; OWASP baseline; secrets managed outside code.          |
| Privacy       | Tenant isolation, data export/delete workflow, configurable retention, no training on customer content without explicit consent.     |
| Availability  | Health checks, horizontal API scaling, queue retries, daily backups with restore drills.                                             |
| Observability | Structured logs with correlation IDs; metrics for latency, job state, provider errors, cost, and token usage; alerting for failures. |
| Accessibility | WCAG 2.2 AA target for web workflows; keyboard navigation and visible focus states.                                                  |
| Performance   | P95 API reads under 500 ms excluding AI jobs; async job progress updates within 5 seconds.                                           |

## 7. AI Safety and Quality Controls

- Use task-specific system prompts and structured output schemas; reject malformed responses before persistence.
- Expose inputs, assumptions, sources, revision history, and uncertainty labels in the UI.
- Run prompt-injection screening on retrieved web content; isolate external content from system instructions.
- Apply rate limits, per-organisation quotas, moderation policies, and spend ceilings.
- Maintain an evaluation set for plan quality, factual grounding, financial arithmetic, and harmful-content handling before changing models or prompts.

## 8. Deployment and Environments

| Environment | Purpose                                       | Controls                                                    |
|-------------|-----------------------------------------------|-------------------------------------------------------------|
| Development | Local feature development.                    | Synthetic data, sandbox providers.                          |
| Staging     | Integration, acceptance, migration rehearsal. | Production-like configuration, limited internal access.     |
| Production  | Customer workloads.                           | Protected deployment, backups, monitoring, least privilege. |

## 9. Test Strategy

- Unit tests for domain calculations, permissions, prompt assembly, and schema validation.
- Integration tests for API-to-database paths, worker retry behaviour, and provider adapters.
- End-to-end tests for onboarding, generation, approval, finance scenario, and export workflows.
- Accessibility checks in CI; load tests for project listing and job dispatch; periodic restore tests.